

Testing Complex Features

Martin Joo

Introduction

In this short case study, I want to show you how you can test complex features. You know, anything outside the standard CRUD functionality. Three features serve as excellent examples:

- Importing subscribers from CSV
- Running automations
- Proceeding a sequence

If you're not familiar with these features, please read them in the book (at least the design chapter) to better understand what's going on. I'm going to test these features with integration or feature tests, so we're not going to mock anything and will actually run everything needed to get the job done. In my opinion, that's the most effective way to test these complex user stories. Without further ado, let's start with the most simple one: importing subscribers from CSV.

Importing Subscribers From CSV

This is the most simple of these three features, so it's a good start. Just as a quick reminder, this is the

`ImportSubscribersAction` class:

```
class ImportSubscribersAction
{
    public static function execute(string $path, User $user): void
    {
        ReadCsvAction::execute($path)
        =>each(function (array $row) use ($user) {
            $parsed = [
                ... $row,
                'tags' => self::parseTags($row, $user),
            ];

            $data = SubscriberData::from($parsed);

            if (self::isSubscriberExist($data, $user)) {
                return;
            }

            UpsertSubscriberAction::execute($data, $user);
        });
    }
}
```

There are four different steps:

- Reading the CSV by calling the `ReadCsvAction`
- Parsing the tags from a comma-separated list.
 - It will parse the tags and create the non-existing ones
- Checking if the subscriber already exists
- Creating the subscriber by calling the `UpsertSubscriberAction`

As the first step, we need to identify the different test cases at a high level:

- Given a CSV with four rows that contain four new subscribers, it will insert four subscribers.
- It will not duplicate existing subscribers.

- Given subscribers with different tags, it will associate the tags with the subscribers.
- It will not duplicate existing tags.

These are the basic test cases. It doesn't seem too hard. The next challenge is: dealing with the CSV. If you look at the `ImportSubscribersAction`, it accepts a `string $path` argument and will read the given path. This is good news because:

- We will create a test CSV file with dummy data in the `storage` directory.
- The test will read this file and will import these subscribers.

As I said earlier, I won't mock anything. If this class reads a CSV with a given format, it's crucial that the test also reads a CSV with the same format as the "production CSV." This way, I'm 100% sure that the action works as expected.

I put this dummy CSV into the `storage/testing/subscribers.csv` path, and this is the content of the file:

email	first_name	last_name	tags
first@example.com	First	Example	Laravel,Vue,Inertia
second@example.com	Second	Example	
third@example.com	Third	Example	Laravel,DDD
fourth@example.com	Fourth	Example	DDD

Make sure that the action can actually read and import the CSV

```
namespace Tests\Fixture\Subscriber;

class ImportSubscribersTest extends TestCase
{
    use RefreshDatabase;

    /** @test */
    public function it_should_import_subscribers()
    {
        $user = User::factory()->create();

        ImportSubscribersAction::execute(storage_path('testing/subscribers.csv'),
        $user);

        $this->assertDatabaseCount('subscribers', 4);
    }
}
```

The first test is not super-specific:

- It calls the action with the testing/subscribers.csv path
- And then checks if the database contains four subscribers after the import

The point of this test is to make sure everything works; we can read the file, the import runs, and there are subscribers in the database after the import. Now, let's get a bit more specific.

Tags are imported and associated with subscribers

```
/** @test */
public function it_should_import_subscribers_with_tags()
{
    $user = User::factory()>create();

    ImportSubscribersAction::execute(storage_path('testing/subscribers.csv'),
    $user);

    $subscriber = Subscriber::whereEmail('first@example.com')>first();
    $this->assertTags($subscriber, ['Laravel', 'Vue', 'Inertia']);

    $subscriber = Subscriber::whereEmail('second@example.com')>first();
    $this->assertTags($subscriber, []);

    $subscriber = Subscriber::whereEmail('third@example.com')>first();
    $this->assertTags($subscriber, ['Laravel', 'DDD']);

    $subscriber = Subscriber::whereEmail('fourth@example.com')>first();
    $this->assertTags($subscriber, ['DDD']);
}
```

In this test, I ensure that every subscriber has the appropriate tags from the CSV. Since I needed to check it four times, I created a helper method called `assertTags`:

```
private function assertTags(Subscriber $subscriber, array $expectedTitles):
void
{
    $this->assertCount(count($expectedTitles), $subscriber->tags);

    foreach ($expectedTitles as $title) {
        $this->assertContains($title, $subscriber->tags->pluck('title'));
    }
}
```

The most important thing about this test is this line:

```
$subscriber = Subscriber::whereEmail('second@example.com')->first();
$this->assertTags($subscriber, []);
```

It makes sure that the importer can handle subscribers with no tags. Empty cells are a common source of errors in CSV or XLS imports. This test only handles non-existing new tags.

It will not duplicate existing subscribers

```
/** @test */
public function it_should_not_duplicate_existing_subscribers()
{
    $user = User::factory()->create();

    Subscriber::factory([
        'email' => 'first@example.com',
    ])->for($user)->create();

    ImportSubscribersAction::execute(storage_path('testing/subscribers.csv'),
    $user);

    $this->assertDatabaseCount('subscribers', 4);
}
```

The difference in this test is that it creates a subscriber with the e-mail `first@example.com` before running the import. This e-mail address is the first one in the CSV. After importing, it makes sure that there are only four subscribers in the database, so:

- An exception was not thrown (duplicate key entry, for example).
- There are four subscribers, so there are no duplications.

It will not duplicate existing tags

```
/** @test */
public function it_should_not_duplicate_existing_tags()
{
    $user = User::factory()>create();

    Tag::factory(['title' => 'Laravel'])>for($user)>create();
    Tag::factory(['title' => 'DDD'])>for($user)>create();

    ImportSubscribersAction::execute(storage_path('testing/subscribers.csv'),
    $user);

    $this->assertDatabaseCount('tags', 4);
}
```

This is very similar to the previous one:

- The test creates two tags, Laravel and DDD. Both of them are in the CSV.
- It runs the import
- After it checks that there are only four tags in the database.

And once again:

- An exception was not thrown (duplicate key entry, for example).
- There are four tags, so there are no duplications (Laravel, Vue, Inertia, DDD).

Now, if you don't like the magic number 4, you can create a constant:

```
class ImportSubscribersTest extends TestCase
{
    use RefreshDatabase;

    private const NUMBER_OF_SUBSCRIBERS_IN_CSV = 4;
    private const NUMBER_OF_TAGS_IN_CSV = 4;
}
```

This way you don't have magic numbers but readable constants.

This is how easy to test a file-related feature such as import. If you look at the action, you can mentally see which test case tests which part of the action.

Running Automations

Now let's move on to a slightly more complicated feature. As a quick reminder, this is how we run automations:

- `SubscribedToFormEvent` is dispatched from the `UpsertSubscriberAction` when a subscriber is created with a form.
- The event listener will dispatch a job called `RunAutomationsJob`
- This job calls the `RunAutomationsAction`, and it will run the automations:

```
class RunAutomationsAction
{
  public static function execute(
    Subscriber $subscriber,
    User $user,
    Events $event
  ): void {
    $automations = Automation::with('steps')
      →whereBelongsTo($user)
      →whereHas('steps', function ($steps) use ($subscriber, $event) {
        $steps
          →whereType(AutomationStepType::Event)
          →whereName($event)
          →where('value→id', $subscriber→form_id);
      })
      →get();

    foreach ($automations as $automation) {
      $steps = $automation
        →steps()
        →whereType(AutomationStepType::Action)
        →get();

      foreach ($steps as $step) {
        $action = Actions::from($step→name)→createAction();
        $action($subscriber, $step);
      }
    }
  }
}
```

```
}
}
```

There are two different kinds of automations:

- `addTags`: it attaches the given tags to the subscriber.
- `addToSequence`: it adds the subscriber to the given sequence.

There are three primary test cases:

- It will add tags.
- It will add the subscriber to a sequence.
- It will do nothing if there's no automation with the subscriber's form.

As you can see, there are jobs and events involved in this test, but fortunately, it's not a problem:

- Events are synchronous.
- Queues can also be synchronous with the correct config. And this is done for you by default in `phpunit.xml`:

```
<php>
  <server name="APP_ENV" value="testing" />
  <server name="CACHE_DRIVER" value="array" />
  <server name="MAIL_MAILER" value="array" />
  <server name="QUEUE_CONNECTION" value="sync" />
  <server name="SESSION_DRIVER" value="array" />
  <server name="TELESCOPE_ENABLED" value="false" />
</php>
```

As you can see, the queue driver is set to sync, meaning everything will work synchronously.

It should add tags

First, let's test the automation that adds tags to the subscriber:

```
class RunAutomationsTest extends TestCase
{
    use RefreshDatabase;

    /** @test */
    public function it_should_add_tags()
    {
        $user = User::factory()>create();
        $form = Form::factory()>create();

        $laravel = Tag::factory(['title' => 'Laravel'])>create();
        $ddd = Tag::factory(['title' => 'DDD'])>create();

        // To be continued
    }
}
```

As the first steps, I create:

- A form. This will be used in the automation itself. It's going to be the "trigger" of the automation, something like "when someone subscribes to **this particular** form add a tag to them."
- Two tags. These two tags are going to be attached to the new subscriber.

The next step is to create the automation itself. We can use a factory to get it done, but it's a bit more complicated and involves more than one factory, and also we need a for each and so on. So instead of using a factory, what about using the `UpsertAutomationAction`? The whole purpose of this action is to create automation and all of its steps from a DTO.

This is an important rule: whenever you need to create some complex data structure in a test or need some mechanics implemented by your "production classes," use them from the tests!

```

public function it_should_add_tags()
{
    $user = User::factory()>create();
    $form = Form::factory()>create();

    $laravel = Tag::factory(['title' => 'Laravel'])>create();
    $ddd = Tag::factory(['title' => 'DDD'])>create();

    UpsertAutomationAction::execute(
        AutomationData::from([
            'name' => 'Test Automation',
            'event' => new AutomationStepData(null, Events::SubscribedToForm->value, $form->id),
            'actions' => [
                new AutomationStepData(null, Actions::AddTag->value, $laravel->id),
                new AutomationStepData(null, Actions::AddTag->value, $ddd->id),
            ]
        ]),
        $user
    );
}

```

Here I'm using two DTOs and the action to create an automation with two steps. It works like this:

- If a subscriber subscribes to `$form`
- It will add the tags `Laravel` and `DDD`

And now we can run this automation. There are two ways to do that.

Calling the `RunAutomationsAction` manually. I have a problem with that. The `UpsertSubscriberAction` contains the most important line of all:

```

if ($data->form && !$data->id) {
    event(new SubscribedToFormEvent($subscriber, $user));
}

```

It will dispatch the event that will trigger the automations.

The second option, of course, is to call the `UpsertSubscriberAction` from the test. I like that one!

```

/** @test */
public function it_should_add_tags()
{
    $user = User::factory()>create();
    $form = Form::factory()>create();

    $laravel = Tag::factory(['title' => 'Laravel'])>create();
    $ddd = Tag::factory(['title' => 'DDD'])>create();

    UpsertAutomationAction::execute(
        AutomationData::from([
            'name' => 'Test Automation',
            'event' => new AutomationStepData(null, Events::SubscribedToForm->value, $form->id),
            'actions' => [
                new AutomationStepData(null, Actions::AddTag->value, $laravel->id),
                new AutomationStepData(null, Actions::AddTag->value, $ddd->id),
            ]
        ]),
        $user
    );

    $subscriber = UpsertSubscriberAction::execute(
        SubscriberData::from([
            'email' => 'test@example.net',
            'first_name' => 'John',
            'last_name' => 'Doe',
            'subscribed_at' => now(),
            'form' => $form,
            'tags' => [],
        ]),
        $user
    );

    $this->assertTrue($subscriber->tags()->where('tags.id', $laravel->id)->exists());
}

```

```
$this->assertTrue($subscriber->tags()->where('tags.id', $ddd->id)->exists());
```

As the last step, it creates the subscriber and asserts that it has the two tags. Once again:

- The `UpsertSubscriberAction` dispatches an event
- The event listener dispatches the `RunAutomationsJob`
- The job calls the `RunAutomationsAction`
- The action instantiates the `AddTag` class
- And finally, the `AddTag` class will add the tags to the subscriber

It should add a subscriber to a sequence

This test is almost the same as the previous one, but now instead of tags, the automation will add the subscriber to a given sequence:

```
/** @test */
public function it_should_add_a_subscriber_to_a_sequence()
{
    $user = User::factory()→create();
    $form = Form::factory()→create();

    $sequence = Sequence::factory()→create();

    UpsertAutomationAction::execute(
        AutomationData::from([
            'name' ⇒ 'Test Automation',
            'event' ⇒ new AutomationStepData(null, Events::SubscribedToForm-
>value, $form→id),
            'actions' ⇒ [
                new AutomationStepData(null, Actions::AddToSequence→value,
$sequence→id),
            ]
        ]),
        $user
    );

    $subscriber = UpsertSubscriberAction::execute(
        SubscriberData::from([
            'email' ⇒ 'test@example.net',
            'first_name' ⇒ 'John',
            'last_name' ⇒ 'Doe',
            'subscribed_at' ⇒ now(),
            'form' ⇒ $form,
            'tags' ⇒ [],
        ]),
        $user
    );
```

```
$this->assertTrue($sequence->subscribers()->where('subscriber_id',
$subscriber->id)->exists());
}
```

Instead of creating tags at the beginning of the test, we need to create a sequence. It doesn't need a particular name or mails or anything like that; we just need the model itself. And as you can see, the automation has an action that uses this sequence. After creating the subscriber, we can check if the `sequence_subscriber` pivot table contains the subscriber.

It should not run automations if the subscriber doesn't have the form.

The next test makes sure that if a subscriber doesn't have a particular `form_id`, automations are not triggered. So in this test, we need to assert that something has not happened. In this case, a `RunAutomationsJob` is being pushed to the queue after creating the subscriber. So if the queue is empty, our class doesn't want to trigger automations, which is good. Fortunately, it's very to mock and assert queues in Laravel:

```
Queue::fake();

// Do something here

Queue::assertNothingPushed();
```

In this case, the `assertNothingPushed` will fail if there's a job in the queue. This is perfect for our test case:


```

/** @test */
public function
it_should_not_run_automations_if_subscriber_doesnt_have_the_form()
{
    $user = User::factory()→create();
    $form = Form::factory()→create();

    $laravel = Tag::factory(['title' ⇒ 'Laravel'])→create();
    $ddd = Tag::factory(['title' ⇒ 'DDD'])→create();

    UpsertAutomationAction::execute(
        AutomationData::from([
            'name' ⇒ 'Test Automation',
            'event' ⇒ new AutomationStepData(null, Events::SubscribedToForm-
>value, $form→id),
            'actions' ⇒ [
                new AutomationStepData(null, Actions::AddTag→value, $laravel→id),
                new AutomationStepData(null, Actions::AddTag→value, $ddd→id),
            ]
        ]),
        $user
    );

    Queue::fake();

    UpsertSubscriberAction::execute(
        SubscriberData::from([
            'email' ⇒ 'test@example.net',
            'first_name' ⇒ 'John',
            'last_name' ⇒ 'Doe',
            'subscribed_at' ⇒ now(),
            // This is an important line
            'form' ⇒ null,
            'tags' ⇒ [],
        ]),
        $user
    );

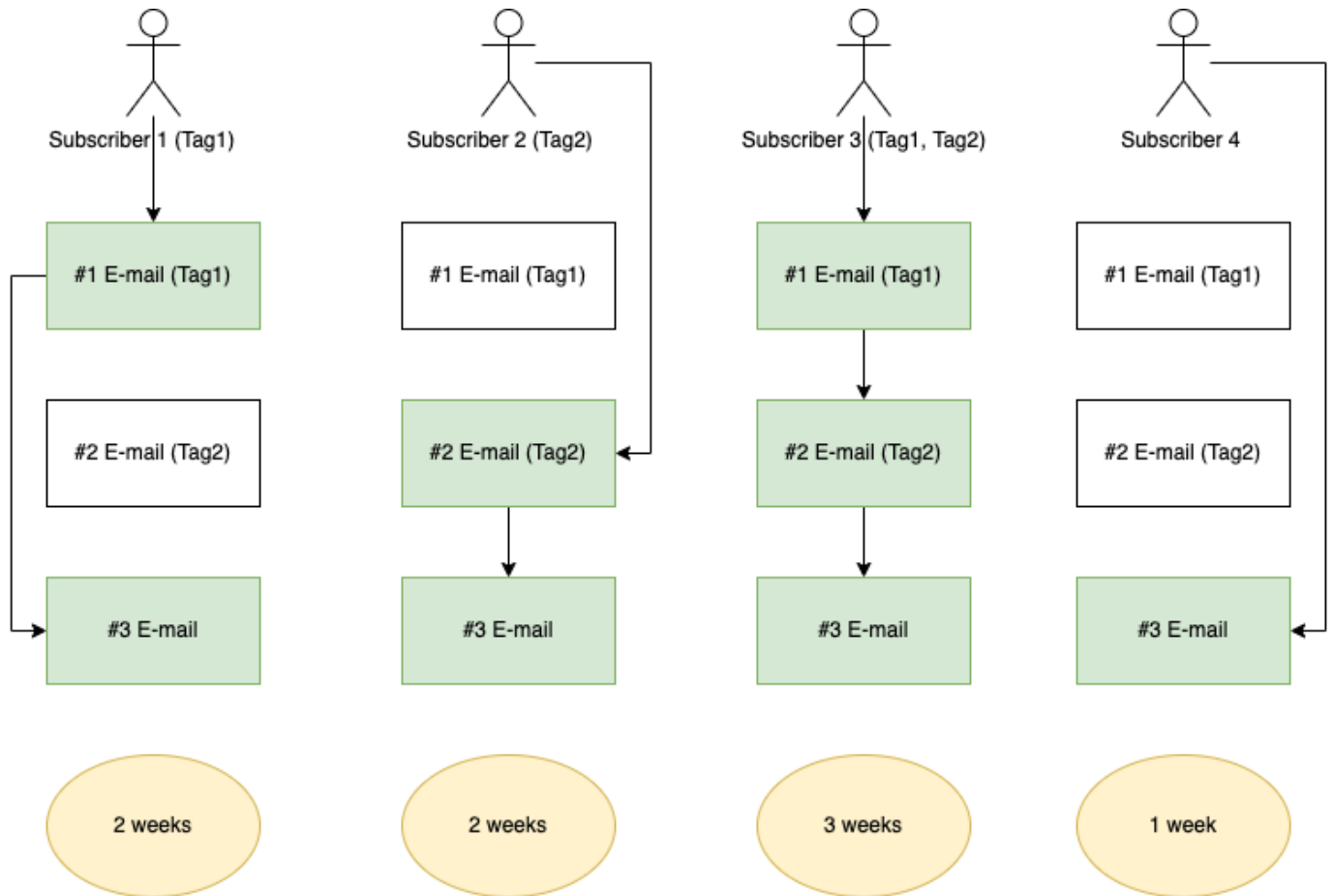
```

```
);  
  
Queue::assertNothingPushed();  
}
```

As you can see, the subscriber doesn't have a form in this test, so we expect no automations to run.

Proceeding Sequences

The last feature I'd like to show you how to test is proceeding a sequence. The feature is quite complicated, so please first read the corresponding chapters in the book. As a quick reminder, here's a flowchart that shows you how subscribers are proceeding in the same sequence:



In this example there are mails in the sequence with the following filters:

- #1 E-mail: Tag1
- #2 E-mail: Tag2
- #3 E-mail: no filters

As you can each subscriber will receive different e-mails based on their tags:

- Subscriber 1: #1 and #3
- Subscriber 2: #2 and #3
- Subscriber 3: every e-mail
- Subscriber 4: only #3

This is what the `ProceedSequenceAction` looks like:

```

class ProceedSequenceAction
{
    /**
     * @var array<int, array<int>>
     */
    private static array $mailsBySubscribers = [];

    public static function execute(Sequence $sequence): void
    {
        self::$mailsBySubscribers = [];

        foreach ($sequence->mails()->wherePublished()->get() as $mail) {
            [$audience, $schedulableAudience] = self::audience($mail);

            self::sendMails($schedulableAudience, $mail, $sequence);

            self::addMailToAudience($audience, $mail);

            self::markAsInProgress($sequence, $schedulableAudience);
        }

        self::markAsCompleted($sequence);
    }
}

```

There are four individual steps:

- Querying the audience
- Sending the e-mails
- Marking subscribers as in progress
- At the end, marking subscribers as completed

There are several test cases we can make here:

- Proceeding subscribers at the same phase. Imagine there are two "Subscriber #1" in the flowchart above.
- Proceeding subscribers at a different phase.
- The action will not send e-mails if it's too early based on the sequence mail's schedule.
- The action will not send e-mails if the current day is turned off in the sequence mail's schedule.
- The action will set the subscriber's status to in progress or completed. We can check this in every test,

so this is not a separate test case.

I think these are the most important ones. Let's start!

It should proceed subscribers at a different phase

```
/** @test */
public function it_should_proceed_subscribers_at_a_different_phase()
{
    $user = User::factory()>create();

    $laravel = Tag::factory([
        'title' => 'Laravel',
    ]>for($user)>create();

    $vue = Tag::factory([
        'title' => 'Vue',
    ]>for($user)>create();

    $laravelSubscriber = Subscriber::factory([
        'first_name' => 'Laravel',
    ]>for($user)>create();

    $vueSubscriber = Subscriber::factory([
        'first_name' => 'Vue',
    ]>for($user)>create();

    $laravelSubscriber->tags()->attach($laravel->id);
    $vueSubscriber->tags()->attach($vue->id);
}
```

In the first few steps, I create subscribers with different tags:

- One subscriber with "Laravel"
- Another one with "Vue"

As the next step, we need a sequence with at least two e-mails:

- One for "Laravel" subscribers.
- Another one for "Vue" subscribers.
- Another one for every subscriber.

```

$sequence = CreateSequenceAction::execute(
    SequenceData::from(Sequence::factory()>for($user)>make()),
    $user
);

$laravelMailFilter = FilterData::from([
    'form_ids' => [],
    'tag_ids' => [$laravel->id],
]);

$laravelMailData = SequenceMail::factory()>for($sequence)>for($user)-
>published()>make();
$laravelMail = UpsertSequenceMailAction::execute(
    SequenceMailData::from([
        ...$laravelMailData->toArray(),
        'filters' => $laravelMailFilters,
        'schedule' => [
            'delay' => 1,
            'unit' => SequenceMailUnit::Hour->value,
            'allowed_days' => SequenceMailScheduleAllowedDaysData::empty(),
        ]
    ]),
    $sequence,
    $user
);

```

Once again, I'm using the upsert actions to save some code. This snippet does the following things:

- It creates a sequence.
- It sets up a `FilterData` DTO with the "Laravel" tag.
- It creates a `published` sequence mail via a factory. Please note that it uses the `make` function, so it's not gonna insert it into the `sequence_mails` table yet.
- And after all that, it actually creates the sequence mail by using the action.

We're going to need almost the same code to create other mails, so I created a helper function called `createMail`. It makes the code a lot better:

```

$sequence = CreateSequenceAction::execute(
    SequenceData::from(Sequence::factory()>for($user)>make()),
    $user
);

$laravelMailFilter = FilterData::from([
    'form_ids' => [],
    'tag_ids' => [$laravel->id],
]);

$vueMailFilter = FilterData::from([
    'form_ids' => [],
    'tag_ids' => [$vue->id],
]);

$laravelMail = $this->createMail($sequence, $user, $laravelMailFilter);
$vueMail = $this->createMail($sequence, $user, $vueMailFilter);
$generalMail = $this->createMail($sequence, $user);

```

So this creates three e-mails:

- One for "Laravel" subscribers.
- Another one for "Vue" subscribers.
- And a last one for every subscriber.

That's all the setup this test needs, so now we can call the `ProceedSequenceAction`:

```
Mail::fake();

ProceedSequenceAction::execute($sequence);

$this->assertDatabaseHas('sent_mails', [
    'sendable_id' => $laravelMail->id,
    'subscriber_id' => $laravelSubscriber->id,
]);

Mail::assertQueued(EchoMail::class, fn (EchoMail $echoMail) =>
    $echoMail->mail->id() === $laravelMail->id
);
```

After executing the action, we need to check two things:

- The `sent_mails` records are created with the correct mail and subscriber.
- The `EchoMail` was queued.

If these two things have happened, the sequence mail has been sent. Now we need to assert these things a lot in this test class, so I created a helper function called `assertMailSent`:

```
private function assertMailSent(SequenceMail $mail, Subscriber $subscriber):
void
{
    $this->assertDatabaseHas('sent_mails', [
        'sendable_id' => $mail->id,
        'subscriber_id' => $subscriber->id,
    ]);

    Mail::assertQueued(EchoMail::class, fn (EchoMail $echoMail) =>
        $echoMail->mail->id() === $mail->id
    );
}
```

Since the "Laravel" subscriber is currently proceeding the sequence but hasn't received every e-mail yet (there's a `$generalMail` with no filters for every subscriber), they need to be marked as in progress:


```

ProceedSequenceAction::execute($sequence);

$this->assertMailSent($laravelMail, $laravelSubscriber);

$this->assertDatabaseHas('sequence_subscriber', [
    'sequence_id' => $sequence->id,
    'subscriber_id' => $laravelSubscriber->id,
    'status' => SubscriberStatus::InProgress,
]);

```

Since we need this assertion a lot, I created a helper method called `assertInProgress`. After that, the test function looks like this:

```

Mail::fake();

ProceedSequenceAction::execute($sequence);

$this->assertMailSent($laravelMail, $laravelSubscriber);
$this->assertMailSent($vueMail, $vueSubscriber);

$this->assertInProgress($sequence, $laravelSubscriber);
$this->assertInProgress($sequence, $vueSubscriber);

```

It's easy and very readable. We are asserting that four things have happened:

- The "Laravel" mail has been sent to the "Laravel" subscriber.
- The "Vue" mail has been sent to the "Vue" subscriber.
- Both subscribers have been marked as "in progress" in `$sequence`

Since every test will use the `assertMailSent` function, which uses the `Mail::assertQueued` method, every test function needs to call `Mail::fake`. We can eliminate that by calling it in the `setUp` lifecycle function:

```
class ProceedSequenceTest extends TestCase
{
    use RefreshDatabase;

    protected function setUp(): void
    {
        parent::setUp();
        Mail::fake();
    }
}
```

This way, the Mail class will be faked in every test.

Now, let's time travel! So the first sequence mail has proceeded. Great, but we also want to check what happens with the second one, right? But each sequence mail has a schedule that states that at least 1 hour needs to pass between mails. So if I call the `ProceedSequenceAction` one more time, nothing will happen. This is where time travel can help us:

```
ProceedSequenceAction::execute($sequence);

$this->assertMailSent($laravelMail, $laravelSubscriber);
$this->assertMailSent($vueMail, $vueSubscriber);

$this->assertInProgress($sequence, $laravelSubscriber);
$this->assertInProgress($sequence, $vueSubscriber);

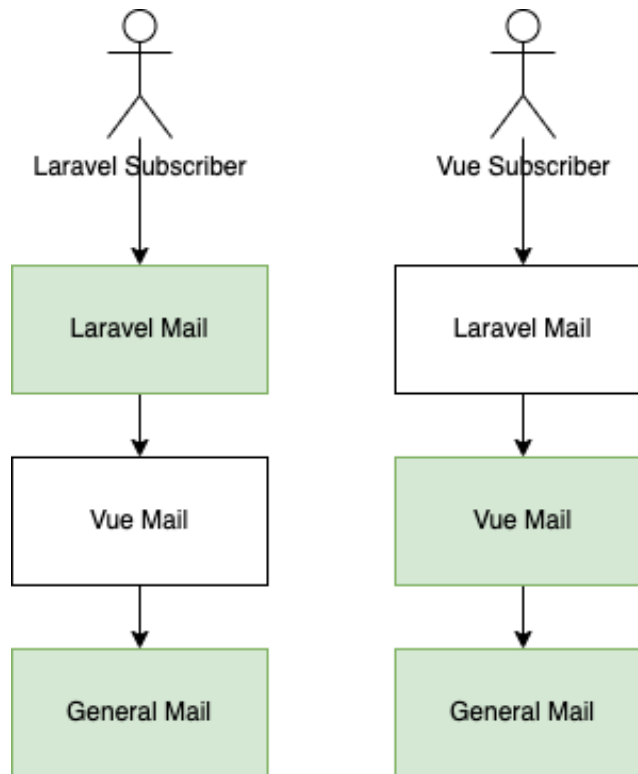
$this->travelTo(now()->addHours(2), function () use ($sequence,
$laravelSubscriber, $vueSubscriber, $generalMail) {
    ProceedSequenceAction::execute($sequence);

    $this->assertMailSent($generalMail, $laravelSubscriber);
    $this->assertMailSent($generalMail, $vueSubscriber);

    $this->assertCompleted($sequence, $laravelSubscriber);
    $this->assertCompleted($sequence, $vueSubscriber);
});
```

The `travelTo` function will set the internal time of Carbon to the value specified in the first argument. Under the hood it calls `Carbon::setTestNow()`. Inside the callback if you call `now()` or `Carbon::now()` you'll get `now()->addHours(2)` because that's the value specified in the first argument. How convenient is that?

After the second execute, I assert that subscribers are completed. Even though there are three mails in the sequence, it only takes two "runs" for these subscribers to finish the whole sequence:



This is what the two "runs" look like:

#	Laravel Subscriber	Vue Subscriber
1	Laravel Mail	Vue Mail
2	General Mail	General Mail

So after running the `ProceedSequenceAction` two times, both subscribers are completed.

It should proceed subscribers at the same phase

The second one will be much easier after building the first test case and creating all of those helpers. In this test, we want two subscribers to receive the same mails at the same time:

```
/** @test */
public function it_should_proceed_subscribers_at_the_same_phase()
{
    $user = User::factory()→create();

    $subscriber1 = Subscriber::factory()→for($user)→create();
    $subscriber2 = Subscriber::factory()→for($user)→create();

    $sequence = CreateSequenceAction::execute(
        SequenceData::from(Sequence::factory()→for($user)→make()),
        $user
    );

    $mail1 = $this→createMail($sequence, $user);
    $mail2 = $this→createMail($sequence, $user);

    ProceedSequenceAction::execute($sequence);

    $this→assertMailSent($mail1, $subscriber1);
    $this→assertMailSent($mail1, $subscriber2);

    $this→assertInProgress($sequence, $subscriber1);
    $this→assertInProgress($sequence, $subscriber2);

    $this→travelTo(now()→addHours(2), function () use ($sequence,
    $subscriber1, $subscriber2, $mail2) {
        ProceedSequenceAction::execute($sequence);

        $this→assertMailSent($mail2, $subscriber1);
        $this→assertMailSent($mail2, $subscriber2);

        $this→assertCompleted($sequence, $subscriber1);
        $this→assertCompleted($sequence, $subscriber2);
    });
}
```

```
});  
}
```

There are no filters or tags; only two subscribers and two mails without filters so that both subscribers will receive these mails. After the second run, they are both completed.

It should not send a mail if it's too early

The next test builds the following scenario:

- There are two subscribers
- And two mails
- The second mail can only be sent after 1 hour has passed since the last mail
- We call the action and assert that the first mail has been sent out to both subscriber
- Immediately after that, we call the action again (without time travel) and assert that no e-mail has been sent
- And both subscribers are "in progress"

This is all because 1 hour has not passed since the two "runs":

```
/** @test */  
public function it_should_not_send_mails_if_its_too_early()  
{  
    $user = User::factory()>create();  
  
    $subscriber1 = Subscriber::factory()>for($user)>create();  
    $subscriber2 = Subscriber::factory()>for($user)>create();  
  
    $sequence = CreateSequenceAction::execute(  
        SequenceData::from(Sequence::factory()>for($user)>make()),  
        $user  
    );  
  
    $mail1 = $this->createMail($sequence, $user);  
    $mail2 = $this->createMail($sequence, $user);  
  
    ProceedSequenceAction::execute($sequence);  
  
    $this->assertMailSent($mail1, $subscriber1);  
    $this->assertMailSent($mail1, $subscriber2);
```

```

ProceedSequenceAction::execute($sequence);

$this->assertMailNotSent($mail2, $subscriber1);
$this->assertMailNotSent($mail2, $subscriber2);

$this->assertInProgress($sequence, $subscriber1);
$this->assertInProgress($sequence, $subscriber2);
}

```

The `assertMailNotSent` is essentially the inversion of `assertMailSent`:

```

private function assertMailNotSent(SequenceMail $mail, Subscriber
$subscriber): void
{
    $this->assertDatabaseMissing('sent_mails', [
        'sendable_id' => $mail->id,
        'subscriber_id' => $subscriber->id,
    ]);

    Mail::assertNotQueued(EchoMail::class, fn (EchoMail $echoMail) =>
        $echoMail->mail->id() === $mail->id
    );
}

```

It should not send mail if it's not the right day

And the last test will check that if the current day is turned off in the sequence mail's schedule then no e-mails will be sent. In this example I will set up a schedule where only Monday is enabled and I will travel to a day which is not Monday. This way if I call the `ProceedSequenceAction` I know that it cannot send e-mails because today is not the day:

```
/** @test */
public function it_should_not_send_mails_if_its_not_the_right_day()
{
    $user = User::factory()→create();

    $subscriber1 = Subscriber::factory()→for($user)→create();
    $subscriber2 = Subscriber::factory()→for($user)→create();

    $sequence = CreateSequenceAction::execute(
        SequenceData::from(Sequence::factory()→for($user)→make()),
        $user
    );

    $mail1Data = SequenceMail::factory()→for($sequence)→for($user)-
>published()→make();
    $mail = UpsertSequenceMailAction::execute(
        SequenceMailData::from([
            ... $mail1Data→toArray(),
            'filters' ⇒ [],
            'schedule' ⇒ [
                'delay' ⇒ 1,
                'unit' ⇒ SequenceMailUnit::Hour→value,
                'allowed_days' ⇒ [
                    'monday' ⇒ true,
                    'tuesday' ⇒ false,
                    'wednesday' ⇒ false,
                    'thursday' ⇒ false,
                    'friday' ⇒ false,
                    'saturday' ⇒ false,
                    'sunday' ⇒ false,
                ],
            ],
        ]),
```

```

    ]
  ]),
  $sequence,
  $user
);

// Not monday
$this->travelTo('2022-03-30', function () use ($sequence, $mail,
$subscriber1, $subscriber2) {
    ProceedSequenceAction::execute($sequence);

    $this->assertMailNotSent($mail, $subscriber1);
    $this->assertMailNotSent($mail, $subscriber2);

    $this->assertNotStarted($sequence, $subscriber1);
    $this->assertNotStarted($sequence, $subscriber2);
});
}

```

After executing the sequence, I check two things:

- No mails were sent.
- Both subscribers are in "not started" status because they haven't received any mails from the sequence yet.

If you don't like the magic date `2022-03-30`, you can use something like this:

```

$this->travelTo('next friday', function () {
    // ...
});

```

This way it's a bit more clear what's going on. However, for me, using a fix date from the past which is not Monday feels more "stable".

Conclusion

So these were the more exciting features from a testing point of view. However, in this essay, my goal was not to reach 100% code coverage but to show you how to test these features and identify the most critical test cases.

As you can usually I'm not going with isolated unit tests for the following reasons:

- These features really test the whole thing. The user story is tested from A to Z. This is very important for me.
- They are easier to write. You don't need to spend your time writing complicated mocks, spies, fakes, and similar things.
- They are more stable. Since these tests work like a "black box," they actually support the development. It means you can refactor anything in your classes; the tests will work without any change. So they actually support the development and refactor process.

However, there are times when you can significantly benefit from isolated unit tests:

- There is a large number of logical branches. For example, you're working on a class that calculates the discounted price of a product, and there are nine different ways of giving a discount to the customers. I'm talking about things like:
 - Percentage-based discount.
 - Fix price discount.
 - A discount can depend on the amount spent by the customer.
 - Or the number of orders in the past.
 - Or the number of products in the order.
 - And several other factors...
- In this situation, you have many inputs and outputs. You have many branches you need to test. Now, in this case, an isolated unit test for this particular class might be the superior solution.

But overall, I believe that you should have mainly high-level feature tests in high-level business applications.